
DEEP LEARNING FOUNDATIONS

NEURAL NETWORKS BY HAND

From Neurons to Backpropagation

A mathematical treatment of feed-forward neural networks, forward propagation, loss functions, computational graphs, backpropagation, gradient descent, and the matrix calculus behind modern deep learning.

PURPOSE

Understand what a neural network actually computes, differentiate it by hand, and connect the scalar chain rule to the vectorized algorithms used in practice.

MATHEMATICAL LEVEL

Graduate mathematics

FOCUS

Deep learning fundamentals

Contents

1	The Objective	3
2	The Artificial Neuron	3
2.1	Weights	4
2.2	Bias	4
3	Activation Functions	4
3.1	ReLU	4
3.2	Sigmoid	4
3.3	Tanh	5
4	Why Nonlinearity Is Necessary	5
5	A Layer	6
5.1	Matrix Dimensions	6
6	Forward Propagation	6
7	Loss Functions	7
8	The Computational Graph	7
9	Complete Single-Neuron Example	8
10	Backpropagation Is Not Gradient Descent	9
11	Why Do We Subtract the Gradient?	9
12	Backpropagation	10
13	Deriving the Matrix Gradient	10
14	Deriving the Hidden-Layer Recurrence	11
15	Complete Two-Layer Network by Hand	11
15.1	Forward Pass	12
15.2	Backward Pass	12
16	The Jacobian Interpretation	13
17	Softmax	14
18	Cross-Entropy	15
19	Mini-Batches	15
20	Optimization Methods	16
20.1	Gradient Descent	16
20.2	Stochastic Gradient Descent	16
20.3	Momentum	16
20.4	RMSProp	16
20.5	Adam	16
21	Numerical Gradient Checking	17

22 Vanishing and Exploding Gradients	17
23 The Complete Neural Network Equations	18
24 How to Solve a Neural Network by Hand	19
25 Final Hand-Solving Exercise	20
26 Core Formula Sheet	21
27 Conceptual Summary	22

1 The Objective

A neural network is ultimately a parameterized function.

Given an input vector $\mathbf{x}$, parameters θ , and target $\mathbf{y}$, the network produces

$$\hat{\mathbf{y}} = f(\mathbf{x}; \theta). \quad (1)$$

A loss function measures the discrepancy between the prediction and the target:

$$\mathcal{L} = \mathcal{L}(\hat{\mathbf{y}}, \mathbf{y}). \quad (2)$$

Training means finding parameters that minimize this loss:

$$\theta^* = \arg \min_{\theta} \mathcal{L}(f(\mathbf{x}; \theta), \mathbf{y}). \quad (3)$$

Everything else in this document exists to answer one question:

How do we calculate the gradient required to change θ ?

For a neural network, this becomes

$$\nabla_{\theta} \mathcal{L}. \quad (4)$$

The central algorithm used to compute this gradient efficiently is **backpropagation**.

The central family of algorithms used to update the parameters from this gradient is **gradient-based optimization**.

These are related, but they are not the same operation.

2 The Artificial Neuron

The simplest neural-network computation is an affine transformation followed by a nonlinear activation.

Given

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}, \quad \mathbf{w} = \begin{bmatrix} w_1 \\ w_2 \\ \vdots \\ w_n \end{bmatrix},$$

the pre-activation is

$$z = \mathbf{w}^T \mathbf{x} + b. \quad (5)$$

Written component-wise,

$$z = \sum_{i=1}^n w_i x_i + b. \quad (6)$$

The neuron then applies an activation function:

$$a = f(z). \quad (7)$$

Therefore,

$$\boxed{a = f(\mathbf{w}^T \mathbf{x} + b)} \quad (8)$$

is the complete mathematical description of a single neuron.

2.1 Weights

Each weight controls the contribution of an input.

For example,

$$z = w_1 x_1 + w_2 x_2 + b.$$

If w_1 is large and positive, increasing x_1 strongly increases z .

If w_1 is negative, increasing x_1 decreases z .

2.2 Bias

The bias shifts the affine transformation:

$$z = \mathbf{w}^T \mathbf{x} + b. \quad (9)$$

Without the bias,

$$z = \mathbf{w}^T \mathbf{x} \quad (10)$$

must pass through the origin.

The bias allows the neuron to translate the transformation.

3 Activation Functions

The activation function is what introduces nonlinearity.

3.1 ReLU

The rectified linear unit is

$$f(z) = \max(0, z). \quad (11)$$

Equivalently,

$$f(z) = \begin{cases} 0, & z < 0, \\ z, & z \geq 0. \end{cases} \quad (12)$$

Its derivative is

$$f'(z) = \begin{cases} 0, & z < 0, \\ 1, & z > 0. \end{cases} \quad (13)$$

The derivative at $z = 0$ is technically undefined. Implementations choose a convention.

3.2 Sigmoid

The sigmoid function is

$$\sigma(z) = \frac{1}{1 + e^{-z}}. \quad (14)$$

Its derivative has the useful form

$$\sigma'(z) = \sigma(z) (1 - \sigma(z)). \quad (15)$$

Therefore, if

$$a = \sigma(z),$$

then

$$\frac{da}{dz} = a(1 - a). \quad (16)$$

3.3 Tanh

The hyperbolic tangent is

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}. \quad (17)$$

Its derivative is

$$\frac{d}{dz} \tanh(z) = 1 - \tanh^2(z). \quad (18)$$

Thus, if $a = \tanh(z)$,

$$\frac{da}{dz} = 1 - a^2. \quad (19)$$

4 Why Nonlinearity Is Necessary

Suppose we stack two purely linear layers:

$$\mathbf{h} = W^{(1)}\mathbf{x} + \mathbf{b}^{(1)} \quad (20)$$

and

$$\mathbf{y} = W^{(2)}\mathbf{h} + \mathbf{b}^{(2)}. \quad (21)$$

Substituting the first equation into the second gives

$$\mathbf{y} = W^{(2)} \left(W^{(1)}\mathbf{x} + \mathbf{b}^{(1)} \right) + \mathbf{b}^{(2)} \quad (22)$$

$$= W^{(2)}W^{(1)}\mathbf{x} + W^{(2)}\mathbf{b}^{(1)} + \mathbf{b}^{(2)}. \quad (23)$$

Define

$$W' = W^{(2)}W^{(1)} \quad (24)$$

and

$$\mathbf{b}' = W^{(2)}\mathbf{b}^{(1)} + \mathbf{b}^{(2)}. \quad (25)$$

Then

$$\mathbf{y} = W'\mathbf{x} + \mathbf{b}'. \quad (26)$$

Therefore, stacking linear layers without nonlinearities does not increase the class of functions representable by the network.

The nonlinear activation prevents this collapse.

5 A Layer

For layer l ,

$$\mathbf{z}^{(l)} = W^{(l)} \mathbf{a}^{(l-1)} + \mathbf{b}^{(l)} \quad (27)$$

and

$$\mathbf{a}^{(l)} = f^{(l)} \left(\mathbf{z}^{(l)} \right). \quad (28)$$

The input is

$$\mathbf{a}^{(0)} = \mathbf{x}. \quad (29)$$

Therefore, a network with L layers is recursively defined by

$$\boxed{\mathbf{z}^{(l)} = W^{(l)} \mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}} \quad (30)$$

followed by

$$\boxed{\mathbf{a}^{(l)} = f^{(l)} \left(\mathbf{z}^{(l)} \right)}. \quad (31)$$

5.1 Matrix Dimensions

If layer $l - 1$ contains n_{l-1} neurons and layer l contains n_l neurons, then

$$W^{(l)} \in \mathbb{R}^{n_l \times n_{l-1}}, \quad (32)$$

$$\mathbf{a}^{(l-1)} \in \mathbb{R}^{n_{l-1}}, \quad (33)$$

and

$$\mathbf{b}^{(l)} \in \mathbb{R}^{n_l}. \quad (34)$$

Consequently,

$$W^{(l)} \mathbf{a}^{(l-1)} \in \mathbb{R}^{n_l}. \quad (35)$$

Shape tracking is not cosmetic. It is one of the easiest ways to detect an incorrect derivation.

6 Forward Propagation

Forward propagation is the process of evaluating the network from input to output.

For a two-layer network:

$$\mathbf{z}^{(1)} = W^{(1)} \mathbf{x} + \mathbf{b}^{(1)}, \quad (36)$$

$$\mathbf{a}^{(1)} = f^{(1)}(\mathbf{z}^{(1)}), \quad (37)$$

$$\mathbf{z}^{(2)} = W^{(2)} \mathbf{a}^{(1)} + \mathbf{b}^{(2)}, \quad (38)$$

$$\hat{\mathbf{y}} = f^{(2)}(\mathbf{z}^{(2)}). \quad (39)$$

The output is therefore a composition:

$$\hat{\mathbf{y}} = f^{(2)} \left(W^{(2)} f^{(1)} \left(W^{(1)} \mathbf{x} + \mathbf{b}^{(1)} \right) + \mathbf{b}^{(2)} \right). \quad (40)$$

Forward propagation computes this function.

Backpropagation differentiates it.

7 Loss Functions

A neural network is not trained merely to produce an output. It is trained to minimize a scalar objective.

For a scalar prediction a and target y , a common loss is squared error:

$$\mathcal{L} = \frac{1}{2}(a - y)^2. \quad (41)$$

The factor $\frac{1}{2}$ is convenient because

$$\frac{\partial \mathcal{L}}{\partial a} = a - y. \quad (42)$$

For a vector prediction,

$$\mathcal{L} = \frac{1}{2} \sum_i (a_i - y_i)^2. \quad (43)$$

Equivalently,

$$\mathcal{L} = \frac{1}{2} \|\mathbf{a} - \mathbf{y}\|_2^2. \quad (44)$$

8 The Computational Graph

Consider the simple computation

$$z = wx + b, \quad (45)$$

$$a = f(z), \quad (46)$$

$$\mathcal{L} = \frac{1}{2}(a - y)^2. \quad (47)$$

The dependency structure is

$$w, x, b \longrightarrow z \longrightarrow a \longrightarrow \mathcal{L}.$$

To find $\partial \mathcal{L} / \partial w$, we follow the path

$$w \rightarrow z \rightarrow a \rightarrow \mathcal{L}.$$

The chain rule gives

$$\boxed{\frac{\partial \mathcal{L}}{\partial w} = \frac{\partial \mathcal{L}}{\partial a} \frac{\partial a}{\partial z} \frac{\partial z}{\partial w}} \quad (48)$$

Since

$$\frac{\partial \mathcal{L}}{\partial a} = a - y, \quad (49)$$

$$\frac{\partial a}{\partial z} = f'(z), \quad (50)$$

and

$$\frac{\partial z}{\partial w} = x, \quad (51)$$

we obtain

$$\boxed{\frac{\partial \mathcal{L}}{\partial w} = (a - y)f'(z)x.} \quad (52)$$

Similarly,

$$\boxed{\frac{\partial \mathcal{L}}{\partial b} = (a - y)f'(z).} \quad (53)$$

This is the fundamental pattern behind backpropagation.

9 Complete Single-Neuron Example

Consider

$$x = 2, \quad w = 0.5, \quad b = 0.2, \quad y = 1. \quad (54)$$

Use a sigmoid activation.

First calculate the pre-activation:

$$z = wx + b \quad (55)$$

$$= (0.5)(2) + 0.2 \quad (56)$$

$$= 1.2. \quad (57)$$

The activation is

$$a = \sigma(1.2) \approx 0.7685. \quad (58)$$

The loss is

$$\mathcal{L} = \frac{1}{2}(a - y)^2 \quad (59)$$

$$= \frac{1}{2}(0.7685 - 1)^2 \quad (60)$$

$$\approx 0.0267. \quad (61)$$

Now differentiate.

First,

$$\frac{\partial \mathcal{L}}{\partial a} = a - y = -0.2315. \quad (62)$$

For the sigmoid,

$$\frac{\partial a}{\partial z} = a(1 - a). \quad (63)$$

Therefore,

$$\frac{\partial a}{\partial z} \approx (0.7685)(0.2315) \approx 0.1779. \quad (64)$$

Thus,

$$\frac{\partial \mathcal{L}}{\partial w} = (-0.2315)(0.1779)(2) \quad (65)$$

$$\approx -0.0824. \quad (66)$$

And

$$\frac{\partial \mathcal{L}}{\partial b} = (-0.2315)(0.1779) \approx -0.0412. \quad (67)$$

The gradient is therefore

$$\nabla \mathcal{L} = \begin{bmatrix} -0.0824 \\ -0.0412 \end{bmatrix}. \quad (68)$$

10 Backpropagation Is Not Gradient Descent

These concepts must be kept separate.

Backpropagation is an algorithm for efficiently computing derivatives of a scalar loss with respect to the parameters of a composed computational graph.

Gradient descent is an optimization procedure that uses those derivatives to update parameters.

Backpropagation produces

$$\nabla_{\theta} \mathcal{L}. \quad (69)$$

Gradient descent then performs

$$\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L} \quad (70)$$

where $\eta > 0$ is the learning rate.

11 Why Do We Subtract the Gradient?

For a differentiable scalar function,

$$\mathcal{L}(\theta + \Delta\theta) \approx \mathcal{L}(\theta) + \nabla_{\theta} \mathcal{L}^T \Delta\theta. \quad (71)$$

Choose

$$\Delta\theta = -\eta \nabla_{\theta} \mathcal{L}. \quad (72)$$

Then

$$\mathcal{L}(\theta + \Delta\theta) \approx \mathcal{L}(\theta) - \eta \nabla_{\theta} \mathcal{L}^T \nabla_{\theta} \mathcal{L} \quad (73)$$

$$= \mathcal{L}(\theta) - \eta \|\nabla_{\theta} \mathcal{L}\|^2. \quad (74)$$

For sufficiently small η ,

$$\mathcal{L}(\theta + \Delta\theta) < \mathcal{L}(\theta). \quad (75)$$

The gradient points in the direction of greatest local increase. Therefore its negative points in the direction of greatest local decrease.

12 Backpropagation

Define the error signal at layer l as

$$\delta^{(l)} = \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{(l)}} \quad (76)$$

where the derivative is understood component-wise.
For the final layer,

$$\delta^{(L)} = \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{(L)}}. \quad (77)$$

The gradient with respect to the weights is

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(l)}} = \delta^{(l)} \left(\mathbf{a}^{(l-1)} \right)^T \quad (78)$$

and the gradient with respect to the bias is

$$\frac{\partial \mathcal{L}}{\partial \mathbf{b}^{(l)}} = \delta^{(l)}. \quad (79)$$

The hidden-layer recurrence is

$$\delta^{(l)} = \left(\left(\mathbf{W}^{(l+1)} \right)^T \delta^{(l+1)} \right) \odot f'(\mathbf{z}^{(l)}) \quad (80)$$

where $\odot$ denotes element-wise multiplication.
This recurrence is the core of backpropagation.

13 Deriving the Matrix Gradient

Consider

$$z_j = \sum_i W_{ji} a_i + b_j. \quad (81)$$

Suppose

$$\delta_j = \frac{\partial \mathcal{L}}{\partial z_j}. \quad (82)$$

Using the chain rule,

$$\frac{\partial \mathcal{L}}{\partial W_{ji}} = \frac{\partial \mathcal{L}}{\partial z_j} \frac{\partial z_j}{\partial W_{ji}}. \quad (83)$$

Since

$$\frac{\partial z_j}{\partial W_{ji}} = a_i, \quad (84)$$

we obtain

$$\frac{\partial \mathcal{L}}{\partial W_{ji}} = \delta_j a_i. \quad (85)$$

Now consider the outer product

$$\delta \mathbf{a}^T. \quad (86)$$

Its (j, i) entry is exactly

$$(\delta \mathbf{a}^T)_{ji} = \delta_j a_i. \quad (87)$$

Therefore,

$$\boxed{\nabla_W \mathcal{L} = \delta \mathbf{a}^T.} \quad (88)$$

The matrix formula is simply the scalar derivative written compactly.

14 Deriving the Hidden-Layer Recurrence

For a hidden layer,

$$\mathbf{z}^{(l+1)} = W^{(l+1)} \mathbf{a}^{(l)} + \mathbf{b}^{(l+1)} \quad (89)$$

and

$$\mathbf{a}^{(l)} = f(\mathbf{z}^{(l)}). \quad (90)$$

The loss depends on $\mathbf{z}^{(l)}$ through $\mathbf{a}^{(l)}$ and $\mathbf{z}^{(l+1)}$.

The derivative first propagates through the next linear layer:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{a}^{(l)}} = \left(W^{(l+1)} \right)^T \delta^{(l+1)}. \quad (91)$$

Then it passes through the activation:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{z}^{(l)}} = \frac{\partial \mathcal{L}}{\partial \mathbf{a}^{(l)}} \odot f'(\mathbf{z}^{(l)}). \quad (92)$$

Therefore,

$$\boxed{\delta^{(l)} = \left(\left(W^{(l+1)} \right)^T \delta^{(l+1)} \right) \odot f'(\mathbf{z}^{(l)}).} \quad (93)$$

The transpose appears because the gradient is being propagated backward through the linear map

$$\mathbf{a}^{(l)} \mapsto W^{(l+1)} \mathbf{a}^{(l)}. \quad (94)$$

15 Complete Two-Layer Network by Hand

Consider a network

$$2 \rightarrow 2 \rightarrow 1.$$

Let

$$\mathbf{x} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}, \quad (95)$$

$$W^{(1)} = \begin{bmatrix} 0.5 & -0.2 \\ 0.3 & 0.8 \end{bmatrix}, \quad \mathbf{b}^{(1)} = \begin{bmatrix} 0.1 \\ -0.1 \end{bmatrix}. \quad (96)$$

Use ReLU in the hidden layer.
For the output layer,

$$W^{(2)} = \begin{bmatrix} 0.4 & -0.6 \end{bmatrix}, \quad b^{(2)} = 0.2. \quad (97)$$

Let the target be

$$y = 0.5. \quad (98)$$

15.1 Forward Pass

First,

$$\mathbf{z}^{(1)} = W^{(1)}\mathbf{x} + \mathbf{b}^{(1)} \quad (99)$$

$$= \begin{bmatrix} 0.5 & -0.2 \\ 0.3 & 0.8 \end{bmatrix} \begin{bmatrix} 1 \\ 2 \end{bmatrix} + \begin{bmatrix} 0.1 \\ -0.1 \end{bmatrix} \quad (100)$$

$$= \begin{bmatrix} 0.5 - 0.4 + 0.1 \\ 0.3 + 1.6 - 0.1 \end{bmatrix} \quad (101)$$

$$= \begin{bmatrix} 0.2 \\ 1.8 \end{bmatrix}. \quad (102)$$

Both values are positive, so

$$\mathbf{a}^{(1)} = \begin{bmatrix} 0.2 \\ 1.8 \end{bmatrix}. \quad (103)$$

Now calculate the output:

$$z^{(2)} = W^{(2)}\mathbf{a}^{(1)} + b^{(2)} \quad (104)$$

$$= (0.4)(0.2) + (-0.6)(1.8) + 0.2 \quad (105)$$

$$= 0.08 - 1.08 + 0.2 \quad (106)$$

$$= -0.8. \quad (107)$$

Assume a linear output activation:

$$\hat{y} = -0.8. \quad (108)$$

The loss is

$$\mathcal{L} = \frac{1}{2}(\hat{y} - y)^2 \quad (109)$$

$$= \frac{1}{2}(-0.8 - 0.5)^2 \quad (110)$$

$$= 0.845. \quad (111)$$

15.2 Backward Pass

Since the output is linear,

$$\frac{\partial \hat{y}}{\partial z^{(2)}} = 1. \quad (112)$$

Therefore,

$$\delta^{(2)} = \hat{y} - y = -1.3. \quad (113)$$

The output weight gradient is

$$\frac{\partial \mathcal{L}}{\partial W^{(2)}} = \delta^{(2)} \left(\mathbf{a}^{(1)} \right)^T \quad (114)$$

$$= -1.3 \begin{bmatrix} 0.2 & 1.8 \end{bmatrix} \quad (115)$$

$$= \begin{bmatrix} -0.26 & -2.34 \end{bmatrix}. \quad (116)$$

The output bias gradient is

$$\frac{\partial \mathcal{L}}{\partial b^{(2)}} = -1.3. \quad (117)$$

Now propagate to the hidden layer:

$$\left(W^{(2)} \right)^T \delta^{(2)} = \begin{bmatrix} 0.4 \\ -0.6 \end{bmatrix} (-1.3) \quad (118)$$

$$= \begin{bmatrix} -0.52 \\ 0.78 \end{bmatrix}. \quad (119)$$

Since both hidden pre-activations are positive,

$$f'(\mathbf{z}^{(1)}) = \begin{bmatrix} 1 \\ 1 \end{bmatrix}. \quad (120)$$

Therefore,

$$\boldsymbol{\delta}^{(1)} = \begin{bmatrix} -0.52 \\ 0.78 \end{bmatrix}. \quad (121)$$

The hidden-layer weight gradient is

$$\frac{\partial \mathcal{L}}{\partial W^{(1)}} = \boldsymbol{\delta}^{(1)} \mathbf{x}^T \quad (122)$$

$$= \begin{bmatrix} -0.52 \\ 0.78 \end{bmatrix} \begin{bmatrix} 1 & 2 \end{bmatrix} \quad (123)$$

$$= \begin{bmatrix} -0.52 & -1.04 \\ 0.78 & 1.56 \end{bmatrix}. \quad (124)$$

The hidden-layer bias gradient is

$$\frac{\partial \mathcal{L}}{\partial \mathbf{b}^{(1)}} = \begin{bmatrix} -0.52 \\ 0.78 \end{bmatrix}. \quad (125)$$

At this point every parameter has a gradient.

16 The Jacobian Interpretation

Backpropagation can be understood as repeated multiplication by Jacobians.

For a vector-valued function

$$\mathbf{y} = f(\mathbf{x}), \quad (126)$$

the Jacobian is

$$J_f = \frac{\partial \mathbf{y}}{\partial \mathbf{x}}. \quad (127)$$

Its (i, j) element is

$$(J_f)_{ij} = \frac{\partial y_i}{\partial x_j}. \quad (128)$$

For a composition

$$\mathbf{y} = f(g(\mathbf{x})), \quad (129)$$

the multivariable chain rule gives

$$\boxed{J_{f \circ g} = J_f J_g}. \quad (130)$$

Backpropagation exploits this structure without explicitly forming every large Jacobian. Instead, it propagates vector-Jacobian products.

If

$$\mathbf{v} = \frac{\partial \mathcal{L}}{\partial \mathbf{y}}, \quad (131)$$

then the gradient with respect to $\mathbf{x}$ is

$$\frac{\partial \mathcal{L}}{\partial \mathbf{x}} = J_g^T J_f^T \mathbf{v}. \quad (132)$$

This is the mathematical basis of reverse-mode automatic differentiation.

17 Softmax

For classification with K classes, the network often produces logits

$$\mathbf{z} = \begin{bmatrix} z_1 \\ \vdots \\ z_K \end{bmatrix}. \quad (133)$$

Softmax converts these into probabilities:

$$\boxed{p_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}} \quad (134)$$

with

$$\sum_i p_i = 1. \quad (135)$$

The probabilities are invariant to adding the same constant to all logits:

$$\text{softmax}(\mathbf{z} + c\mathbf{1}) = \text{softmax}(\mathbf{z}). \quad (136)$$

For numerical stability, one commonly computes

$$p_i = \frac{e^{z_i - m}}{\sum_j e^{z_j - m}}, \quad m = \max_j z_j. \quad (137)$$

18 Cross-Entropy

For a one-hot target $\mathbf{y}$, the cross-entropy loss is

$$\mathcal{L} = - \sum_{i=1}^K y_i \log p_i. \quad (138)$$

Substitute softmax:

$$p_i = \frac{e^{z_i}}{\sum_j e^{z_j}}. \quad (139)$$

Therefore,

$$\mathcal{L} = - \sum_i y_i \log \left(\frac{e^{z_i}}{\sum_j e^{z_j}} \right) \quad (140)$$

$$= - \sum_i y_i z_i + \sum_i y_i \log \left(\sum_j e^{z_j} \right). \quad (141)$$

Since

$$\sum_i y_i = 1, \quad (142)$$

we obtain

$$\mathcal{L} = - \sum_i y_i z_i + \log \left(\sum_j e^{z_j} \right). \quad (143)$$

Differentiate with respect to z_k :

$$\frac{\partial \mathcal{L}}{\partial z_k} = -y_k + \frac{e^{z_k}}{\sum_j e^{z_j}} \quad (144)$$

$$= -y_k + p_k. \quad (145)$$

Therefore,

$$\frac{\partial \mathcal{L}}{\partial z_k} = p_k - y_k. \quad (146)$$

In vector form,

$$\nabla_{\mathbf{z}} \mathcal{L} = \mathbf{p} - \mathbf{y}. \quad (147)$$

This simplification is one of the most important derivatives in modern neural-network training.

19 Mini-Batches

Suppose a batch contains m examples.

The average loss is

$$\mathcal{L}_{\text{batch}} = \frac{1}{m} \sum_{i=1}^m \mathcal{L}_i. \quad (148)$$

By linearity of differentiation,

$$\nabla_{\theta} \mathcal{L}_{\text{batch}} = \frac{1}{m} \sum_{i=1}^m \nabla_{\theta} \mathcal{L}_i. \quad (149)$$

Thus the batch gradient is the mean of the individual gradients.

Mini-batch training approximates the gradient of the full dataset while reducing computational cost per update.

20 Optimization Methods

20.1 Gradient Descent

The basic update is

$$\boxed{\theta_{t+1} = \theta_t - \eta \nabla_{\theta} \mathcal{L}(\theta_t).} \quad (150)$$

20.2 Stochastic Gradient Descent

Instead of computing the gradient over the entire dataset, SGD uses one example or a small mini-batch:

$$\theta_{t+1} = \theta_t - \eta \widehat{\nabla_{\theta} \mathcal{L}}. \quad (151)$$

The estimate is noisy but usually much cheaper.

20.3 Momentum

Momentum maintains a velocity variable:

$$\mathbf{v}_t = \beta \mathbf{v}_{t-1} + \nabla_{\theta} \mathcal{L}_t. \quad (152)$$

The parameters are updated using

$$\theta_{t+1} = \theta_t - \eta \mathbf{v}_t. \quad (153)$$

20.4 RMSProp

RMSProp maintains an exponential moving average of squared gradients:

$$\mathbf{s}_t = \beta \mathbf{s}_{t-1} + (1 - \beta) (\nabla_{\theta} \mathcal{L}_t)^2. \quad (154)$$

The update is

$$\theta_{t+1} = \theta_t - \eta \frac{\nabla_{\theta} \mathcal{L}_t}{\sqrt{\mathbf{s}_t} + \epsilon}. \quad (155)$$

The square and division are element-wise.

20.5 Adam

Adam combines momentum-like first moments with second moments.

$$\mathbf{m}_t = \beta_1 \mathbf{m}_{t-1} + (1 - \beta_1) \mathbf{g}_t, \quad (156)$$

$$\mathbf{v}_t = \beta_2 \mathbf{v}_{t-1} + (1 - \beta_2) \mathbf{g}_t^2. \quad (157)$$

Bias-corrected estimates are

$$\hat{\mathbf{m}}_t = \frac{\mathbf{m}_t}{1 - \beta_1^t}, \quad (158)$$

$$\hat{\mathbf{v}}_t = \frac{\mathbf{v}_t}{1 - \beta_2^t}. \quad (159)$$

The update is

$$\theta_{t+1} = \theta_t - \eta \frac{\hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{v}}_t + \epsilon}}. \quad (160)$$

21 Numerical Gradient Checking

Analytical derivatives can be checked numerically.

For a scalar parameter θ ,

$$\frac{\partial \mathcal{L}}{\partial \theta} \approx \frac{\mathcal{L}(\theta + \epsilon) - \mathcal{L}(\theta - \epsilon)}{2\epsilon} \quad (161)$$

This is the central finite-difference approximation.

The approximation error is

$$O(\epsilon^2) \quad (162)$$

under suitable smoothness assumptions.

For a neural network, one can compare

$$g_{\text{analytic}} \quad (163)$$

with

$$g_{\text{numeric}}. \quad (164)$$

A useful relative error measure is

$$\text{error} = \frac{|g_{\text{analytic}} - g_{\text{numeric}}|}{\max(1, |g_{\text{analytic}}|, |g_{\text{numeric}}|)}. \quad (165)$$

Gradient checking is particularly useful when implementing backpropagation manually.

22 Vanishing and Exploding Gradients

Backpropagation repeatedly multiplies derivatives.

Schematically,

$$\frac{\partial \mathcal{L}}{\partial \mathbf{h}^{(1)}} = J_L J_{L-1} \cdots J_2 \frac{\partial \mathcal{L}}{\partial \mathbf{h}^{(L)}}. \quad (166)$$

If the typical magnitude of the Jacobians is less than one, repeated multiplication can make gradients approach zero:

$$\|\nabla \mathcal{L}\| \rightarrow 0. \quad (167)$$

If their magnitude is greater than one, gradients may grow rapidly:

$$\|\nabla \mathcal{L}\| \rightarrow \infty. \quad (168)$$

This is the mathematical origin of vanishing and exploding gradients.

Sigmoid and tanh can contribute to vanishing gradients because their derivatives are bounded:

$$0 < \sigma'(z) \leq \frac{1}{4}. \quad (169)$$

For ReLU,

$$f'(z) \in \{0, 1\} \quad (170)$$

away from zero, which avoids repeatedly multiplying by a small positive derivative in the active region.

23 The Complete Neural Network Equations

A feed-forward neural network can be summarized by the forward recurrence

$$\mathbf{z}^{(l)} = W^{(l)} \mathbf{a}^{(l-1)} + \mathbf{b}^{(l)} \quad (171)$$

and

$$\mathbf{a}^{(l)} = f^{(l)} \left(\mathbf{z}^{(l)} \right). \quad (172)$$

At the output,

$$\hat{\mathbf{y}} = \mathbf{a}^{(L)}. \quad (173)$$

The loss is

$$\mathcal{L} = \mathcal{L}(\hat{\mathbf{y}}, \mathbf{y}). \quad (174)$$

Define

$$\boldsymbol{\delta}^{(l)} = \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{(l)}}. \quad (175)$$

Then

$$\nabla_{W^{(l)}} \mathcal{L} = \boldsymbol{\delta}^{(l)} \left(\mathbf{a}^{(l-1)} \right)^T \quad (176)$$

and

$$\nabla_{\mathbf{b}^{(l)}} \mathcal{L} = \boldsymbol{\delta}^{(l)}. \quad (177)$$

For hidden layers,

$$\boldsymbol{\delta}^{(l)} = \left(\left(W^{(l+1)} \right)^T \boldsymbol{\delta}^{(l+1)} \right) \odot f'(\mathbf{z}^{(l)}). \quad (178)$$

Finally, optimization performs

$$W^{(l)} \leftarrow W^{(l)} - \eta \nabla_{W^{(l)}} \mathcal{L} \quad (179)$$

and

$$\boxed{\mathbf{b}^{(l)} \leftarrow \mathbf{b}^{(l)} - \eta \nabla_{\mathbf{b}^{(l)}} \mathcal{L}.}$$
 (180)

This is the complete mathematical loop:

$$\boxed{\text{Forward pass} \rightarrow \text{Loss} \rightarrow \text{Backpropagation} \rightarrow \text{Gradient} \rightarrow \text{Parameter update}.}$$
 (181)

24 How to Solve a Neural Network by Hand

When solving a network manually, follow this sequence.

Step 1: Write the dimensions

For every layer, record

$$n_0, n_1, \dots, n_L.$$
 (182)

Then determine

$$W^{(l)} \in \mathbb{R}^{n_l \times n_{l-1}}.$$
 (183)

Step 2: Forward pass

For every layer calculate

$$\mathbf{z}^{(l)} = W^{(l)} \mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}$$
 (184)

followed by

$$\mathbf{a}^{(l)} = f^{(l)}(\mathbf{z}^{(l)}).$$
 (185)

Do not skip intermediate values.

Step 3: Calculate the loss

Write the exact loss function.

For squared error,

$$\mathcal{L} = \frac{1}{2} \|\hat{\mathbf{y}} - \mathbf{y}\|^2.$$
 (186)

Step 4: Start at the output

Calculate

$$\boldsymbol{\delta}^{(L)} = \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{(L)}}.$$
 (187)

Step 5: Calculate the parameter gradients

For each layer,

$$\nabla_{W^{(l)}} \mathcal{L} = \boldsymbol{\delta}^{(l)} \left(\mathbf{a}^{(l-1)} \right)^T.$$
 (188)

$$\nabla_{\mathbf{b}^{(l)}} \mathcal{L} = \boldsymbol{\delta}^{(l)}.$$
 (189)

Step 6: Propagate the error backward

For hidden layers,

$$\delta^{(l)} = \left(\left(W^{(l+1)} \right)^T \delta^{(l+1)} \right) \odot f'(\mathbf{z}^{(l)}). \quad (190)$$

Step 7: Update parameters

With learning rate η ,

$$\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}. \quad (191)$$

25 Final Hand-Solving Exercise

Consider the network

$$2 \rightarrow 2 \rightarrow 1.$$

Let

$$\mathbf{x} = \begin{bmatrix} 2 \\ -1 \end{bmatrix}, \quad (192)$$

$$W^{(1)} = \begin{bmatrix} 0.4 & -0.7 \\ 0.2 & 0.5 \end{bmatrix}, \quad (193)$$

$$\mathbf{b}^{(1)} = \begin{bmatrix} 0.1 \\ -0.2 \end{bmatrix}, \quad (194)$$

$$W^{(2)} = \begin{bmatrix} 0.6 & -0.3 \end{bmatrix}, \quad (195)$$

and

$$b^{(2)} = 0.1. \quad (196)$$

Use ReLU in the hidden layer and a linear output.

Let

$$y = 0.8. \quad (197)$$

Calculate, by hand:

1. $\mathbf{z}^{(1)}$
2. $\mathbf{a}^{(1)}$
3. $z^{(2)}$
4. $\hat{y}$
5. $\mathcal{L}$
6. $\delta^{(2)}$
7. $\nabla_{W^{(2)}} \mathcal{L}$
8. $\nabla_{b^{(2)}} \mathcal{L}$

9. $\delta^{(1)}$

10. $\nabla_{W^{(1)}} \mathcal{L}$

11. $\nabla_{b^{(1)}} \mathcal{L}$

Finally perform one gradient-descent update using

$$\eta = 0.1. \quad (198)$$

The complete solution should contain every intermediate value.

26 Core Formula Sheet

Neuron

$$z = \mathbf{w}^T \mathbf{x} + b \quad (199)$$

$$a = f(z) \quad (200)$$

Layer

$$\mathbf{z}^{(l)} = W^{(l)} \mathbf{a}^{(l-1)} + \mathbf{b}^{(l)} \quad (201)$$

$$\mathbf{a}^{(l)} = f^{(l)}(\mathbf{z}^{(l)}) \quad (202)$$

Squared Error

$$\mathcal{L} = \frac{1}{2} \|\hat{\mathbf{y}} - \mathbf{y}\|^2 \quad (203)$$

Error Signal

$$\delta^{(l)} = \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{(l)}} \quad (204)$$

Weight Gradient

$$\nabla_{W^{(l)}} \mathcal{L} = \delta^{(l)} \left(\mathbf{a}^{(l-1)} \right)^T \quad (205)$$

Bias Gradient

$$\nabla_{b^{(l)}} \mathcal{L} = \delta^{(l)} \quad (206)$$

Hidden-Layer Backpropagation

$$\delta^{(l)} = \left(\left(W^{(l+1)} \right)^T \delta^{(l+1)} \right) \odot f'(\mathbf{z}^{(l)}) \quad (207)$$

Softmax

$$p_i = \frac{e^{z_i}}{\sum_j e^{z_j}} \quad (208)$$

Cross-Entropy

$$\mathcal{L} = - \sum_i y_i \log p_i \quad (209)$$

Softmax + Cross-Entropy Gradient

$$\nabla_{\mathbf{z}} \mathcal{L} = \mathbf{p} - \mathbf{y} \quad (210)$$

Gradient Descent

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} \mathcal{L} \quad (211)$$

27 Conceptual Summary

A neural network is a composition of parameterized functions:

$$f = f^{(L)} \circ f^{(L-1)} \circ \dots \circ f^{(1)}. \quad (212)$$

Each layer performs an affine transformation followed by a nonlinearity:

$$\mathbf{a}^{(l)} = f^{(l)} \left(W^{(l)} \mathbf{a}^{(l-1)} + \mathbf{b}^{(l)} \right). \quad (213)$$

The forward pass evaluates this composition.

The loss converts the final output into a scalar:

$$\mathcal{L} = \mathcal{L}(\hat{\mathbf{y}}, \mathbf{y}). \quad (214)$$

Backpropagation applies the chain rule in reverse:

$$\frac{\partial \mathcal{L}}{\partial \theta} = \frac{\partial \mathcal{L}}{\partial u_n} \frac{\partial u_n}{\partial u_{n-1}} \dots \frac{\partial u_1}{\partial \theta}. \quad (215)$$

For neural networks, the resulting derivatives can be organized into error signals:

$$\boldsymbol{\delta}^{(l)} = \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{(l)}}. \quad (216)$$

Those error signals give the parameter gradients:

$$\nabla_{W^{(l)}} \mathcal{L} = \boldsymbol{\delta}^{(l)} \left(\mathbf{a}^{(l-1)} \right)^T. \quad (217)$$

Finally, an optimizer changes the parameters:

$$\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}. \quad (218)$$

Thus the entire training mechanism can be reduced to

compose $\rightarrow$ evaluate $\rightarrow$ differentiate $\rightarrow$ update

The important conceptual distinction is:

$$\boxed{\text{Neural network} = \text{function composition}} \quad (219)$$

$$\boxed{\text{Backpropagation} = \text{efficient reverse-mode differentiation}} \quad (220)$$

$$\boxed{\text{Gradient descent} = \text{parameter optimization using the gradient}} \quad (221)$$

Understanding these three statements is the mathematical core of training neural networks.